CSC311: Introduction to Machine Learning

Project Report

Characterizing Student Perceptions of Generative AI Models

Submitted by

Caroline Lyu, Izabelle Marianne, Kevin Hu, Victoria Pergola

University of Toronto
Fall 2025

1 Executive Summary

We explored three model families—Softmax Regression, Random Forest, and Neural Networks—to classify student perceptions of three generative AI models (ChatGPT, Claude, Gemini) based on survey responses containing mixed ordinal categorical, multi-select categorical, and free-text features. After comprehensive hyperparameter tuning using GroupKFold cross-validation with student-level grouping, Neural Networks achieved the best performance with 70.3% validation accuracy. This model outperformed others by effectively learning non-linear feature interactions in the high-dimensional space (2000 TF-IDF features + 20 categorical features) through multiple hidden layers.

2 Data Exploration

2.1 Dataset Overview

The dataset contains 825 survey responses from students (275 responses per model, indicated balanced class labels) with each student contributing one response per model. Features include Single-select ordinal categorical features, about usage and trust. Multi-select categorical features requires selection to describe strengths and weaknesses of the model and free-text response features to describe suboptimal responses verification process. The target variable (**target**) is categorical and the name of model being reviewed.

2.2 Missing Data Analysis

Missing data was mainly found in three features **How_often_do_you_expect_this_model_to_provide_res** had substantial missing data with 62 rows missing values, followed by **Which_types_of_tasks_do_you_feel_this_model_handles_best** and **For_which_types_of_tasks_do_you_feel_this_model_tends_to_give_suboptimal_responses?** with 22 and 11 missing values respectively. The free-text fields showed modest missingness. This pattern of missingness informed our preprocessing imputation strategies.

2.3 Feature-Specific Analysis

Most informative feature was likelihood of use for academic tasks and frequency of response verification, which showed notable skew for ChatGPT, potentially indicating different trust levels across models.

2.4 Data Preprocessing

We applied several transformations appropriate to each feature type using scikit-learn pipelines for reproducibility and proper handling of train-test separation.

Text Features: Each text column was preprocessed separately using TF-IDF. Raw text was converted into numerical vectors based on how frequently a term appears on the document (each row) relative to how unique it is across the dataset (each text column)

Ordinal Features: Ordinal encoding was used to preserve the inherent ordering and simple median imputation was used as it is less influenced by outliers. KNN imputation was avoided since our other features had high dimensionality, which is not suitable for distance based algorithms.

Multi-select Categorical Features: For multi-select features were processed using a `MultiSelectBinari` transformer that reliably handles the comma-separated task selections (prevented incorrectly splitting on internal commas). Each of the 8 possible task categories was converted into a binary indicator variable, resulting in 16 total binary features, avoiding assumptions about any ordering between task types. Missing values in these columns were im-

puted as zeros (task not selected) because the absence of any selection holds meaningful information

2.5 Data Splitting and Leakage Prevention

We employed a 70-30 train-test split (approximately 576 training samples, 249 test samples) based on guidance for relatively small datasets. We split at the student level rather than the response level, so that all responses from a student remained in some split to avoid leakage across splits due to shared student-level patterns or writing styles. The test set was only used for the final evaluation of our models and was never accessed during data exploration, feature engineering, model selection, or hyperparameter tuning phases.

3 Methodology

We explored three distinct model families, each offering different strengths for our mixed-feature classification task with categorical and text data.

3.1 Random Forest

Random Forest is an ensemble bagging model that takes the majority prediction form multiple description trees as the final prediction. Each tree is trained on a bootstrap sample of the training data and considers only a random subset of features at each split. This injected randomization reduces variance in the final prediction, making the model less likely to overfit compared to a single decision tree.

This model works well for our task, since it can handle various data types without scaling or normalization. It is also resistant to noise due to the randomization techniques and can capture non-linear patterns and interactions without requiring explicit feature engineering.

3.2 Neural Network

A neural network model makes use of interconnected layers and non-linear activation functions to produce non-linear feature maps of the input feature vectors. The final softmax layer then produces a probability distribution for all classes for prediction. This model is well suited to our high-dimensional text features and the potential non-linear interactions between text, ratings, and categorical inputs. Neural networks handle such complexity effectively and can learn feature interactions directly.

3.3 Softmax Regression

Softmax regression extends binary logistic regression to multi-class classification using softmax function. We included this model family as a baseline to assess whether simpler linear relationships could adequately capture the classification patterns, or whether the additional complexity of tree-based ensemble methods was necessary. Softmax does well with balanced class distribution found in our dataset, but has difficulty handling the high-dimensional sparse feature space created by one-hot encoding and TFIDF, and also cannot capture non-linear feature interactions without explicit feature engineering.

3.4 Hyperparameter Tuning

For each model, a two-stage optimization approach was used: first, the TF-IDF hyperparameters were optimized while keeping model parameters at their default values, then the model hyperparameters were optimized using the best TF-IDF configuration.

Hyperparameter tuning for each stage was further divided into two more steps. First, randomized search was employed to efficiently explore a broad hyperparameter space and

identify promising value ranges. Second, grid search was performed over these narrowed ranges to conduct a thorough, granular evaluation and select optimal hyperparameters. Both stages utilized GroupKFold cross-validation with 5 folds, where unique student IDs served as groups to ensure all samples from the same student appeared in the same fold, preventing data leakage between training and validation sets. Each fold allocated 80% of the data for training and 20% for validation. For each fold, a new model was trained on the training set and evaluated on the validation set, with final hyperparameters selected based on the highest mean cross-validation accuracy across all folds.

3.4.1 TF-IDF Tuning

Hyperparameter searching was performed using grid search for each vectorizer. The candidate values were informed by analyzing the distribution and length of the text vectors (e.g., vectors with longer text required a higher minimum `max_features`). For each vectorizer, the configuration that achieved the best cross-validated accuracy was selected.

We tuned the TF–IDF representation by varying the vocabulary size (`tfidf_max_features` $\in \{300, 500, 800\}$) to find informative and low sparse vocabiulary size. And also tuned (`tfidf_ngram_range` $\in \{(1,1),(1,2)\}$) to evaluate whether incorporating short phrase patterns improved feature expressiveness.

3.4.2 Random Forest Tuning

We performed grid search over the following hyperparameters, chosen to balance ensemble stability, depth-related generalization, feature sampling strategy, and node-splitting behaviour. `n_estimators`: {170, 180, 200} and `max_depth`: {40, 50, 60} — were varied across these ranges to to balance capturing non-linear structure and overfitting. `max_features`: {`''sqrt''`, 0.25, 0.5, 0.75, 1.0} checked for how much the model should ensure individuals trees are uncorrelated. Tuning `min_samples_split`: {2, 5, 10} and {2, 3, 4, 5} and `min_samples_leaf`: {1, 2, 4} and {1, 2} helped balance between the model's ability to capture fine-grained patterns and generalize well. Finally `bootstrap`: {`True`, `False`} checked the effect of sampling with and without replacement.

3.4.3 Multilayer Perceptrons

As this model had the widest range of parameters to test, two additional features were implemented: Early stopping, monitoring validation loss at the end of each epoch, to halt training if no improvement occurred for three consecutive epochs. We used the Weights and Biases (`wandb`) dashboard to easily monitor experiments. Each sweep consisted of a configuration of the following values: Learning rate (`lr`): {0.01, 0.001, 0.0001}: which tested for the trade-off between fast convergence and optimization stability. `batch_size`: {16, 32, 64} — balanced computational cost with gradient-update stability. `hidden_units`: {128, 256, 512}: helped compared lower-capacity models (less overfitting) with higher-capacity ones (more expressive). `dropout_rate`: {0.3, 0.5, 0.7} helped test a range of regularization strengths. Finally, `epochs`: was fixed at 20 to simply serve as an upper bound, with early stopping determining actual training length.

3.4.4 Softmax Regression

We varied the regularization strength, `C`: {0.001, 0.01, 0.1, 1.0} to test how the model should penalize large weight, `max_iter`: {5000, 10000, 20000} tested for the appropriate number of iterations to ensure the optimizer converged (especially for the high dimensional TF-IDF space). Finally, `class_weight`: {'balanced} was used to ensure the model penalizes mistakes on all classes equally.

3.4.5 Training

Random forests train by building multiple decision trees, and follow a greedy pattern. The feature split for each tree tries to minimize an impurity measure. We used Gini as our measure since it is computationally efficient.

Neural networks make use of gradient-based optimization, which consists of a forward pass to calculate prediction and activation, a loss calculation, and backpropagation to compute gradients. These gradients are used to update current weights, which relies on an optimization algorithm. We used the ADAM optimizer to update our weights, the option used in the majority of machine learning frameworks and models, as it can dynamically adapt learning rates per parameter and makes use of momentum for a more efficient training process.

We recognized neural networks require large amounts of data for efficient training and generalizability. To prevent overfitting, we made use of early stopping and dropout regularization techniques as mentioned before. Early stopping ensures training is stopped when validation loss no longer decreases, preventing overfitting. Dropout randomly drops a subset of neurons to prevent the network from relying too much on any single neuron, again reducing overfitting.

3.5 Evaluation Metrics

We made use of two evaluation metrics to assess model performance:

Accuracy: This was our primary measure of accuracy, and measures the proportion of correct predictions across all classes. It does not perform well for imbalanced classes but given our balanced class distribution, accuracy serves as a reasonable primary metric.

F1-Score: This is computed by average the F1-score across all classes, which gives equal weight to each class's performance. The F1 score balances the trade-off between false positives and false negatives, making it particularly informative when both types of errors are equally costly. Additionally, the F1-score is more robust than accuracy to potential class imbalances that might arise in our cross-validation folds.

3.6 Implementation Details

We implemented all models using Python 3.10 with the following libraries: scikit-learn for Random Forests, Softmax Regression, and hyperparameter tuning (Grid Search and

Randomized Search). PyTorch was used for constructing and training the neural network, and Weights & Biases experiment tracking was used to determine the best weights for the NN. We also implemented custom PyTorch training loops, early stopping and custom Python functions to perform text and categorical preprocessing, word frequency analysis and stop word filtering. All experiments were run with fixed random seeds for reproducibility.

4 Results

4.1 Model Comparison

Table 1 shows cross-validation performance for all three models. The Neural Network model achieved the highest mean cross fold validation accuracy (70.3%) and F1-score of 69.3, beating out Random Forest (67.5%) and Softmax Regression (67.5%) and thus was chosen as the final model and used in prediction script.

Table 1: Cross-validation performance (mean over 5 folds)

Model	Accuracy (%)	F1-Score (%)
Neural Network	70.3	69.3
Random Forest	67.5	67.2
Softmax Regression	67.5	67.2

Table 2, 3 and 4 in the Appendix show test set evaluation metrics and class level performance for all models. Across all three models, ChatGPT had highest recall while Gemini had the lowest. Additionally, the confusion matrices (Figures 1 to 3 in the Appendix) show clear patterns for misclassifications: Claude and Gemini are the most frequently confused pair, while ChatGPT was the easiest label to identify. This suggests overlap in the student reviews for Gemini and Claude

4.2 Final Model: Neural Network

Best hyperparameters: `batch_size`=64, `dropout_rate`=0.3, epochs=20 (early stopping), `hidden_units`=256, `learning_rate`=0.0001.

4.3 Test Performance Estimate: 0.7108.

We chose the neural network as our final model because it achieved the highest cross-validation accuracy and F1-score. Even though the softmax regression model showed slightly better test performance, we still proceeded with the neural network because it demonstrated more stable and consistent performance overall.

Using the best hyperparameter configuration we obtained from the GroupKFold cross-validation, we trained a final neural network on the full training set to use with our test set. This model gave a test accuracy of 0.7108.

The justification for using this value as our final test accuracy is for the following reasons: Our final test set accuracy (71.08%) matches our validation metrics (70.3% accuracy, 69.3% F1-score), which indicates that the model generalizes well and is not overfit to any particular data split. This was due to the regularization techniques such as early stopping and dropout.

5 Contributions and Learning

Caroline Lyu: Wrote script for neural network and its hyperparameter tuning. Wrote data exploration and methodology section of final report.

Kevin Hu: Conversion of preprocessing and neural net scripts to numpy implementation, neural net tuning.

Izabelle Marianne: Wrote script for random forest and logistic regression and their hyperparameter tuning. Wrote methodology and results section of final report.

Victoria Pergola: Explored and evaluated multiple model families. Conducted comprehensive hyperparameter tuning experiments. Authored final report.

6 Appendix

Table 2: Neural Network Performance (Test accuracy = 0.7108, Test Macro F1 = 0.7054)

Class	Precision	Recall	F1-score
ChatGPT	0.76	0.89	0.82
Claude	0.71	0.67	0.69
Gemini	0.64	0.57	0.60

Table 3: Random Forest Performance (Test accuracy = 0.7108, Test Macro F1 = 0.7073)

Class	Precision	Recall	F1-score
ChatGPT	0.77	0.87	0.81
Claude	0.71	0.60	0.65
Gemini	0.65	0.66	0.65

Table 4: Softmax Regression Performance (Test accuracy = 0.7149, Test Macro F1 = 0.7120)

Class	Precision	Recall	F1-score
ChatGPT	0.76	0.86	0.81
Claude	0.73	0.69	0.71
Gemini	0.64	0.60	0.62

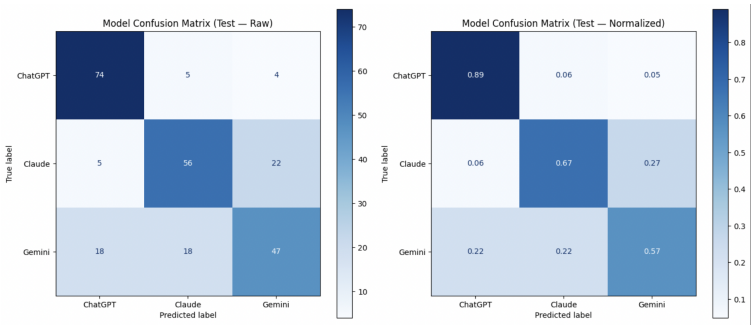

Figure 1: Confusion Matrix — Neural Network

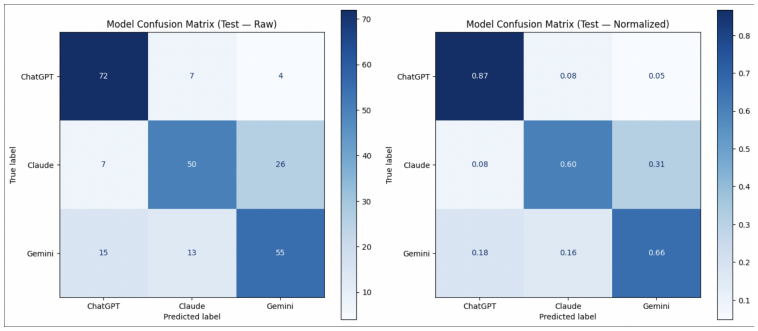

Figure 2: Confusion Matrix — Random Forest

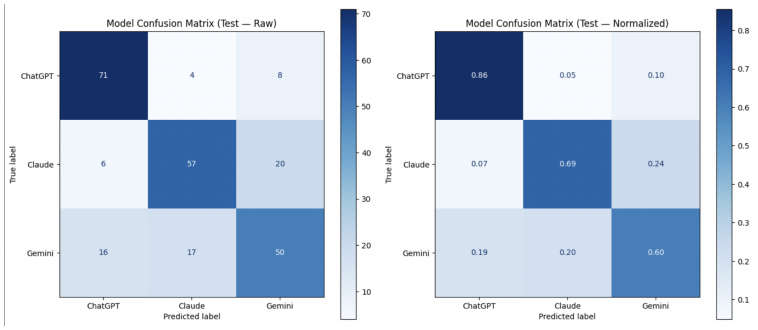

Figure 3: Confusion Matrix — Softmax Regression